Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering

2026-2027
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3

(25CS2103E)

Multi-Pattern Unicode Search Engine Using Aho–Corasick Algorithm

Kandula Poojitha Rani	**2520030460**
Himesh	**2520030317**
Ishwak	**2520030458**

COURSE INSTRUCTOR

Swati
Professor
Department of Computer Science and Engineering

Multi-Pattern Unicode Search Engine Using Aho–Corasick Algorithm

1. Introduction

A search engine is a software application that searches for specific words, phrases, or patterns in a given text. Traditional searching methods usually search for one pattern at a time. However, in many real-world applications, it is necessary to search for multiple patterns simultaneously. For example, a document scanning system may need to identify several keywords, a cybersecurity application may search for multiple suspicious strings, and a text-processing application may need to detect words written in different languages.

The **Multi-Pattern Unicode Search Engine Using the Aho–Corasick Algorithm** is a system designed to find multiple patterns in a text efficiently. It uses the Aho–Corasick string-matching algorithm, which can search for many patterns in a single pass through the input text.

Unicode is a universal character-encoding standard that supports text from almost every writing system, including English, Telugu, Hindi, Tamil, Arabic, Chinese, and many others. By supporting Unicode, the proposed search engine can process multilingual text, special symbols, emojis, and other characters beyond the basic ASCII character set.

For example, consider a text containing the following sentence:

"Artificial Intelligence is useful. కృత్రిమ మేధస్సు is the Telugu term for Artificial Intelligence."

Suppose the search engine must find the patterns "Artificial", "Intelligence", and "కృత్రిమ". Instead of searching for each pattern separately, the Aho–Corasick algorithm can identify all occurrences during a single traversal of the text.

This project combines efficient pattern matching with Unicode text processing to provide a fast, accurate, and flexible search solution.

2. Objectives of the Case Study

The main objective of this case study is to develop an efficient search engine capable of identifying multiple patterns in Unicode text.

The specific objectives are:

Multi-pattern searching: Search for several keywords or phrases simultaneously.
Unicode support: Handle text in multiple languages and scripts, including Telugu, Hindi, English, and other Unicode-supported languages.
Improved performance: Reduce unnecessary repeated scanning by processing the text in a single pass after building the search structure.
Accurate matching: Identify the starting and ending positions of matched patterns.

Efficient data structures: Use a trie, failure links, and output links to perform pattern matching.
Practical applicability: Support applications such as document analysis, spam detection, log monitoring, content filtering, and multilingual information retrieval.

3. Overview of the Aho–Corasick Algorithm

The Aho–Corasick algorithm is a well-known string-matching algorithm developed by Alfred V. Aho and Margaret J. Corasick in 1975. It is particularly useful when a large collection of patterns must be searched for within a single text.

The algorithm combines the concepts of a trie and finite-state matching.

A **trie** is a tree-like data structure used to store strings. Each edge represents a character, and a path from the root represents a pattern or a prefix of a pattern.

The algorithm mainly uses three components:

Trie structure: Stores all the patterns in a shared tree, allowing common prefixes to be reused.
Failure links: Indicate the next state to follow when the current character does not extend the present match.
Output information: Records which patterns end at a particular state, including patterns that are suffixes of other matched patterns.

Working procedure

The algorithm works in two major phases.

Phase 1: Pattern preprocessing

All input patterns are inserted into a trie. After insertion, failure links are constructed using breadth-first search (BFS). These links allow the search to continue without restarting from the root every time a mismatch occurs.

Phase 2: Text searching

The search engine processes the input text from left to right. For each character, it follows trie transitions or failure links until it reaches a suitable state. Whenever a state contains one or more output patterns, the corresponding matches are reported.

The important advantage is that the text does not need to be scanned independently for every pattern.

4. Unicode Support in the Search Engine

Unicode support is one of the most important features of the proposed system. ASCII-based search engines are limited to a small set of characters, whereas Unicode supports characters from many languages and writing systems.

For example, the following strings contain characters from different scripts:

English: `hello`
Telugu: నమస్కారం
Hindi: नमस्ते
Tamil: வணக்கம்
Arabic: مرحبا
Emoji: 😊

A Unicode-aware search engine should be capable of processing these strings without assuming that every character occupies one byte.

In UTF-8, a Unicode character may be represented by one to four bytes. Therefore, the implementation must choose a consistent unit for matching and reporting positions.

A suitable approach is to decode the input into Unicode code points and build the trie using those code points. The engine can then report match positions as Unicode code-point indices. If byte offsets are required for file operations, the system can additionally maintain a mapping from code-point positions to UTF-8 byte offsets.

Unicode also introduces normalization concerns. Some visually identical text can have different underlying code-point sequences. For example, accented characters may be represented as a single precomposed character or as a base character followed by a combining mark. Unicode normalization, such as NFC, can help make equivalent representations consistent.

However, normalization should be applied consistently to both the patterns and the text. Case folding, accent handling, and language-specific equivalence rules should be optional features because their correct behavior depends on the application's requirements.

5. System Architecture

The proposed search engine consists of the following modules:

5.1 Input module

This module accepts the text to be searched and a collection of patterns from the user or an external file. The patterns can contain English words, regional-language text, special symbols, or emojis.

5.2 Unicode preprocessing module

This module decodes the input into Unicode code points. If configured, it also performs normalization or case folding on both the patterns and the input text.

5.3 Trie construction module

The trie construction module inserts all patterns into a common tree. Shared prefixes are stored only once, reducing unnecessary duplication.

5.4 Failure-link construction module

This module uses BFS to calculate failure links. These links help the algorithm recover from partial matches efficiently.

5.5 Pattern-matching module

This module scans the text and identifies all occurrences of the stored patterns. When a match is found, it records the matched pattern and its position.

5.6 Result generation module

The result module displays the matching patterns, their occurrence counts, and their starting and ending positions. Depending on the application, results may also be exported to a file or used by another program.

The overall workflow is:

Input text and patterns → Unicode preprocessing → Trie construction → Failure-link generation → Text scanning → Match reporting

6. Example 1: Searching for Multiple English Words

Consider the input text:

```
ushers are using he and she.
```

Let the set of patterns be:

```
{he, she, his, hers}
```

The algorithm inserts these four patterns into a trie and builds the failure links.

During the text scan, it discovers the following matches:

Pattern	Starting index	Ending index
she	1	3
he	2	3
hers	2	5
he	15	16
she	23	25
he	24	25

The table uses zero-based character positions and inclusive ending indices. The matches are listed according to their positions in the text.

This example demonstrates an important feature of Aho–Corasick: **multiple patterns can match at overlapping positions**. For example, the text `she` contains both `she` and `he`. The algorithm can report both matches rather than selecting only the longest pattern.

The exact result positions depend on the complete input text, so an implementation should calculate positions directly instead of relying on manually specified indices.

7. Example 2: Searching Unicode Text in Multiple Languages

Consider the following input text:

```
నమస్కారం! Welcome. నమస్కారం to everyone.
```

Suppose the search patterns are:

```
{నమస్కారం, Welcome, everyone}
```

The Unicode-aware search engine searches for all three patterns in the same input text.

The expected matches are:

Pattern	Number of occurrences
నమస్కారం	2
Welcome	1
everyone	1

The search engine identifies the Telugu word నమస్కారం twice, even though it is written in a non-Latin script. It also finds the English words `Welcome` and `everyone`.

This example demonstrates that the algorithm is not restricted to English alphabets. When implemented using Unicode code points, it can process text from multiple scripts.

For accurate multilingual searching, the engine should also define how punctuation, whitespace, normalization, and case sensitivity are handled.

8. Example 3: Detecting Suspicious Words in Messages

A practical application of the search engine is spam or suspicious-message detection.

Suppose a messaging platform wants to check incoming messages for the following patterns:

{free prize, claim now, winner, urgent}

An incoming message contains:

Congratulations! You are a winner. Claim now to receive your free prize.

The engine searches the message once and identifies the patterns that occur in it.

The detected results include:

```
winner
claim now
free prize
```

The application can use these results to flag the message for further review.

This approach can be more efficient than running a separate search operation for each suspicious phrase, especially when the pattern dictionary contains thousands of entries. In a production system, the detected phrases should be treated as indicators rather than definitive proof of spam.

9. Time and Space Complexity

The efficiency of the Aho–Corasick algorithm is one of its main advantages.

Let: P be the total number of characters or code points across all patterns.N be the length of the input text.Z be the total number of reported matches.

The approximate time complexity is:O(P+N+Z)

Here, O(P) represents pattern insertion and preprocessing, O(N) represents scanning the text, and O(Z)represents the work required to report all matches.

The Z term is important because a text may contain a large number of overlapping or repeated matches. Reporting these results necessarily takes additional time.

The space complexity is approximately O(P)for a trie with failure links and associated pattern information, excluding any separately stored input text and output list. The exact memory usage depends on the transition representation. Sparse maps or dictionaries can be useful for Unicode because the possible character set is very large, while a dense transition table can consume excessive memory.

For a large text and a large collection of patterns, Aho–Corasick is generally more efficient than searching for every pattern independently.

10. Advantages of the Proposed System

The proposed Unicode search engine offers several advantages:

Fast searching: Multiple patterns are searched for in one traversal of the text.

Multilingual compatibility: Unicode code-point processing supports a wide variety of scripts.
Overlapping match detection: The engine can report multiple patterns that end at the same position.
Reusable preprocessing: Once the trie and failure links are built, the same pattern collection can be reused for multiple text inputs.
Flexible applications: The system can be used for document scanning, message filtering, log analysis, and keyword detection.
Scalability: The algorithm can handle large pattern collections, subject to available memory and the number of matches produced.

11. Limitations and Challenges

Despite its advantages, the proposed system has some limitations.

First, building the trie and failure links requires preprocessing time and memory. If only one very short pattern needs to be searched in a small text, a simpler algorithm may be sufficient.

Second, Unicode processing requires careful handling of encoding, normalization, and character positions. A search engine that compares raw UTF-8 bytes without considering the intended matching rules may produce incorrect or confusing results for some multilingual applications.

Third, the number of reported matches can become very large when patterns overlap heavily. The application may therefore need limits, filtering, or streaming output.

Finally, Aho–Corasick performs exact pattern matching by default. It does not automatically handle spelling mistakes, synonyms, transliteration, or semantic similarity. These capabilities require additional processing techniques.

12. Applications

The Multi-Pattern Unicode Search Engine can be used in several real-world applications:

Search engines: Finding multiple keywords in documents or web pages.
Cybersecurity: Detecting known malicious strings or suspicious indicators in logs.
Spam detection: Identifying messages containing known spam phrases.
Digital libraries: Searching multilingual books, articles, and archived documents.
Content moderation: Detecting configured words and phrases across supported languages.
Log monitoring: Finding several error codes or diagnostic messages in system logs.
Educational tools: Searching vocabulary and phrases across English and regional-language learning materials.
Data mining: Extracting predefined terms from large collections of textual data.

13. Conclusion

The Multi-Pattern Unicode Search Engine Using the Aho–Corasick Algorithm provides an efficient approach to searching for multiple patterns in text. By combining a trie, failure links,

and output information, the algorithm avoids repeatedly scanning the entire text for each individual pattern.

Unicode support extends the usefulness of the engine beyond ASCII-based applications, enabling the processing of multilingual content, special symbols, and emojis. Proper Unicode decoding, consistent normalization, and well-defined position reporting are essential for producing reliable results.

The case study demonstrates how the algorithm can be applied to English keyword searching, Telugu text searching, and suspicious-message detection. Its time complexity of approximately O(P+N+Z)makes it suitable for many large-scale text-processing tasks.

In conclusion, this system is a practical solution for applications that require fast, accurate, and multilingual multi-pattern searching. Future improvements could include a graphical user interface, file and document upload support, configurable case-insensitive matching, language-aware normalization, and integration with real-time log-monitoring or content-filtering systems.

ALGORITHM / PSEUDO CODE

1. Overview of the Algorithm

TextHack is a Java-based text searching and analysis engine that integrates advanced algorithms from six modules of the Data Structures and Algorithms syllabus. Its primary objective is to search multiple patterns in text documents efficiently and provide additional functionalities such as single-pattern searching, fuzzy searching, document comparison, graph optimization, scheduling, primality testing, and prefix scanning.

The system follows a menu-driven approach. The user can add documents, select an operation, provide the required input, and view the result. Each operation invokes a corresponding algorithm.

The central algorithm is Aho–Corasick, which searches for multiple patterns simultaneously using a trie and failure links. Other algorithms extend the system's functionality and demonstrate different problem-solving techniques.

2. Step-by-Step Algorithm

Step 1: Initialize the System

1. Start the application.
2. Initialize the document storage structure.
3. Set the initial document count to zero.
4. Display the TextHack menu.
5. Wait for the user to select an operation.

Step 2: Add and Store Documents

1. Ask the user to enter a document name and its text.
2. Read the document content until the designated termination marker is entered.

3. Store the document in the available memory.

4. Increase the document count.

5. Display a confirmation message.

6. Return to the main menu.

The documents are stored in memory in the current implementation. They are available for searching and comparison while the program is running.

Step 3: Perform Unicode Multi-Pattern Search Using Aho–Corasick

This is the main algorithm of TextHack.

1. Accept multiple search patterns from the user.

2. Create an empty trie.

3. Insert every pattern into the trie character by character.

4. Mark the ending node of each pattern as a terminal node.

5. Construct failure links using a breadth-first traversal.

6. Process the stored document from beginning to end.

7. Follow trie transitions for matching characters.

8. If a transition is unavailable, follow failure links until a valid transition is found or the root is reached.

9. Whenever a terminal node is reached, report the corresponding pattern or patterns.

10. Continue until the entire document has been processed.

11. Display the matching patterns and their positions, as supported by the implementation.

Purpose: To find multiple patterns efficiently without performing a completely separate search for each pattern.

Step 4: Perform Single-Pattern Search Using KMP

1. Accept a text document and a single search pattern.

2. Calculate the Longest Proper Prefix that is also a Suffix (LPS) table for the pattern.

3. Initialize text and pattern indices.

4. Compare the current text character with the current pattern character.

5. If the characters match, advance both indices.

6. If the entire pattern matches, record its starting position.

7. If a mismatch occurs, use the LPS table to determine how much of the pattern can be reused.

8. Continue until the text has been processed.

9. Display the matching positions or report that no match was found.

Purpose: To search for one pattern efficiently while avoiding unnecessary repeated comparisons.

Step 5: Perform Fuzzy Search Using Edit Distance

1. Accept a search query and the stored documents.

2. Extract candidate words or strings according to the implementation.

3. Create a dynamic-programming table for the query and each candidate.

4. Initialize the first row and column with the costs of insertions and deletions.

5. For each pair of characters, calculate the minimum cost of insertion, deletion, or substitution.

6. Obtain the final edit distance from the last cell of the table.

7. Compare the distance with the program's matching criteria.

8. Display the sufficiently similar candidates and their associated documents.

Purpose: To identify approximate matches when the search query contains spelling differences or minor character changes.

Step 6: Compare Two Documents Using LCS

1. Ask the user to select two stored documents.
2. Represent their text as sequences.
3. Create a dynamic-programming table for the two sequences.
4. Initialize the first row and column to zero.
5. If the current elements match, extend the common subsequence by one.
6. Otherwise, select the maximum of the relevant neighboring table values.
7. Read the final table value to obtain the longest common subsequence length.
8. Use this value to calculate or report a similarity measure, if implemented.
9. Display the comparison result.

Purpose: To measure shared sequence content between two documents.

Step 7: Execute Graph Algorithms

Maximum Flow — Edmonds–Karp

1. Initialize the predefined graph and its capacity values.
2. Identify the source and destination vertices.
3. Use breadth-first search to find an augmenting path in the residual graph.
4. Determine the minimum available capacity along that path.
5. Increase the flow along the path and update residual capacities.
6. Repeat until no augmenting path exists.
7. Display the maximum flow value.

Bipartite Matching

1. Initialize the two sets of vertices and their allowed connections.
2. Start with an empty matching.

3. Search for an augmenting path that can improve the current matching.

4. Update the matching when such a path is found.

5. Repeat until no additional augmenting path can be found.

6. Display the resulting matching.

These graph operations use predefined examples in the current implementation.

Step 8: Perform Optimization and Scheduling

Vertex-Cover Approximation

1. Initialize the graph and an empty vertex-cover set.

2. Select an uncovered edge.

3. Add both endpoints of the edge to the cover.

4. Remove all edges incident to the selected endpoints.

5. Repeat until no uncovered edges remain.

6. Display the approximate vertex cover.

Scheduling Demonstration

1. Initialize the predefined set of tasks and their deadlines.

2. Arrange tasks according to the implemented deadline-based rule.

3. Apply the scheduling heuristic.

4. Display the resulting task order or schedule.

The scheduling result depends on the specific rule used by the implementation and is not necessarily optimal for every scheduling objective.

Step 9: Perform Miller–Rabin Primality Testing

1. Accept an integer from the user.

2. Handle small integers and obvious composite cases.

3. Express the number minus one as an odd factor multiplied by a power of two.

4. Select a test base.

5. Calculate the required modular exponentiation.

6. Check whether the result satisfies the Miller–Rabin conditions.

7. Repeat for additional bases when required.

8. Report whether the number is composite or probably prime, subject to the implementation's guarantees.

Step 10: Perform Prefix Scan

1. Accept a sequence of numbers.

2. Initialize the prefix-scan operation.

3. Combine sequence elements using the chosen associative operation, such as addition.

4. Calculate the cumulative result for each position.

5. Store the resulting prefix values.

6. Display the prefix-scan output.

The current implementation demonstrates the scan algorithm's structure sequentially rather than executing operations concurrently on multiple processors.

Step 11: Terminate the Application

1. Display the menu after each completed operation.

2. Accept the next user choice.

3. Continue processing until the user selects Exit.

4. Terminate the program.

3. Overall Pseudocode

```text
ALGORITHM TEXT HACK

BEGIN
    Initialize document storage
    documentCount ← 0

    REPEAT
        Display TextHack menu
        Read userChoice

        SWITCH userChoice

            CASE 0:
                Display system and API overview

            CASE 1:
                Read document name and content
                Store document in memory
                Update documentCount

            CASE 2:
                Display all stored documents

            CASE 3:
                IF documentCount = 0 THEN
                    Display "Add documents first"
```

ELSE

Read search patterns

Build Aho-Corasick trie

Construct failure links

Search stored documents

Display matching patterns

END IF

CASE 4:

Read a single search pattern

Construct KMP LPS table

Search the selected text

Display matching positions

CASE 5:

Read fuzzy search query

Calculate edit distances

Select sufficiently similar matches

Display results

CASE 6:

Select two documents

Calculate LCS using dynamic programming

Display comparison results

CASE 7:

Initialize flow network

Execute Edmonds-Karp

Display maximum flow

CASE 8:

Initialize bipartite graph

Execute bipartite matching

Display matching result

CASE 9:

Initialize graph

Execute vertex-cover approximation

Display approximate cover

CASE 10:

Initialize scheduling example

Apply scheduling heuristic

Display schedule

CASE 11:

Read an integer

Execute Miller-Rabin test

Display primality result

CASE 12:

Read numerical sequence

Execute prefix scan

Display prefix results

CASE 13:

Terminate application

```
        DEFAULT:
            Display "Invalid option"

    END SWITCH

  UNTIL userChoice = 13

END
```

4. Expected Outcome

The TextHack application provides a unified environment for demonstrating multiple advanced algorithms. Its main text-processing features support multi-pattern searching, single-pattern searching, approximate matching, and document comparison. Additional modules demonstrate graph algorithms, approximation, scheduling, randomized primality testing, and prefix scanning.

The project illustrates how different algorithms can be selected according to the problem being solved. Its modular, menu-driven structure also allows future improvements, including persistent database storage, additional string algorithms, performance comparisons, and integration of document relationships into graph-based analysis.

CODE

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;

public class Main {

   static BufferedReader in = new BufferedReader(
         new InputStreamReader(System.in));

   // Shared document store used by TextHack.
   static final int MAX_DOCS = 100;
   static String[] documents = new String[MAX_DOCS];
   static int docCount = 0;

   static String read(String prompt) throws IOException {
      System.out.print(prompt);
      String s = in.readLine();
      return s == null ? "" : s;
   }

   static int number(String prompt) throws IOException {
      return Integer.parseInt(read(prompt).trim());
   }

   static String multiline(String prompt) throws IOException {
      System.out.println(prompt);
      System.out.println("Type END on a separate line to finish.");

      StringBuilder b = new StringBuilder();
      String line;
```

```
    while ((line = in.readLine()) != null) {
        if (line.equals("END")) break;
        if (b.length() > 0) b.append('\n');
        b.append(line);
    }

    return b.toString();
}

static int[] points(String s) {
    return s.codePoints().toArray();
}

static void addDocument(String text) {
    if (docCount == MAX_DOCS) {
        System.out.println("Document store is full.");
        return;
    }

    documents[docCount] = text;
    System.out.println("Document added with ID: " + docCount);
    docCount++;
}

static void showDocuments() {
    if (docCount == 0) {
        System.out.println("No documents stored.");
        return;
    }

    for (int i = 0; i < docCount; i++) {
        System.out.println("\nDocument " + i + ":");
        System.out.println(documents[i]);
```

```
    }
}

  MODULE 2: AHO-CORASICK MULTI-PATTERN SEARCH
// Custom trie, failure links, output lists and BFS queue.


static class Edge {
    int cp;
    Node target;
    Edge next;

    Edge(int cp, Node target, Edge next) {
        this.cp = cp;
        this.target = target;
        this.next = next;
    }
}

static class Output {
    int id;
    Output next;

    Output(int id, Output next) {
        this.id = id;
        this.next = next;
    }
}

static class Node {
    Edge edges;
    Node failure;
    Output outputs;
```

```
    Node get(int cp) {
      for (Edge e = edges; e != null; e = e.next)
        if (e.cp == cp) return e.target;
      return null;
    }

    void addEdge(int cp, Node target) {
      edges = new Edge(cp, target, edges);
    }

    void addOutput(int id) {
      outputs = new Output(id, outputs);
    }
  }

  static class AhoCorasick {
    Node root = new Node();
    Node[] queue;
    String[] patterns;
    int[][] patternPoints;
    int nodes = 1;

    AhoCorasick(String[] patterns) {
      this.patterns = patterns;
      patternPoints = new int[patterns.length][];

      for (int i = 0; i < patterns.length; i++) {
        if (patterns[i] == null || patterns[i].isEmpty())
          throw new IllegalArgumentException(
              "Patterns cannot be empty.");

        patternPoints[i] = points(patterns[i]);
```

```
    }

    buildTrie();
    buildFailures();
}

void buildTrie() {
    for (int id = 0; id < patterns.length; id++) {
        Node current = root;

        for (int cp : patternPoints[id]) {
            Node next = current.get(cp);

            if (next == null) {
                next = new Node();
                current.addEdge(cp, next);
                nodes++;
            }

            current = next;
        }

        current.addOutput(id);
    }
}

void buildFailures() {
    queue = new Node[nodes];
    int head = 0, tail = 0;

    for (Edge e = root.edges; e != null; e = e.next) {
        e.target.failure = root;
        queue[tail++] = e.target;
```

```
    }

    while (head < tail) {
      Node current = queue[head++];

      for (Edge e = current.edges; e != null; e = e.next) {
        int cp = e.cp;
        Node child = e.target;
        Node fallback = current.failure;

        while (fallback != root && fallback.get(cp) == null)
          fallback = fallback.failure;

        Node target = fallback.get(cp);
        child.failure = target == null ? root : target;

        for (Output o = child.failure.outputs;
          o != null; o = o.next)
          child.addOutput(o.id);

        queue[tail++] = child;
      }
    }
  }

  int search(String text, int docId) {
    int[] t = points(text);
    Node current = root;
    int count = 0;

    for (int i = 0; i < t.length; i++) {
      int cp = t[i];
```

```
            while (current != root && current.get(cp) == null)
                current = current.failure;

            Node next = current.get(cp);
            current = next == null ? root : next;

            for (Output o = current.outputs;
                 o != null; o = o.next) {
                int start = i - patternPoints[o.id].length + 1;

                System.out.println(
                        "Document " + docId +
                        " | Pattern: " + patterns[o.id] +
                        " | Start: " + start +
                        " | End: " + i);

                count++;
            }
        }

        return count;
    }
}

static void multiPatternSearch() throws IOException {
    if (docCount == 0) {
        System.out.println("Add documents first.");
        return;
    }

    int n = number("Number of patterns: ");
    if (n < 1 || n > 500) {
        System.out.println("Enter between 1 and 500 patterns.");
```

```
            return;
        }

        String[] patterns = new String[n];

        for (int i = 0; i < n; i++)
            patterns[i] = read("Pattern " + (i + 1) + ": ");

        AhoCorasick engine = new AhoCorasick(patterns);
        int total = 0;

        for (int d = 0; d < docCount; d++)
            total += engine.search(documents[d], d);

        System.out.println("Total matches: " + total);
        System.out.println("Algorithm: Aho-Corasick");
        System.out.println("Search complexity: O(N + Z), after preprocessing");
    }

      MODULE 2: KMP SINGLE-PATTERN SEARCH

    static int kmp(String text, String pattern) {
        int[] t = points(text);
        int[] p = points(pattern);

        if (p.length == 0) {
            System.out.println("Pattern cannot be empty.");
            return 0;
        }

        int[] lps = new int[p.length];

        for (int i = 1, len = 0; i < p.length;) {
```

```
        if (p[i] == p[len]) {
            lps[i++] = ++len;
        } else if (len > 0) {
            len = lps[len - 1];
        } else {
            lps[i++] = 0;
        }
    }

    int i = 0, j = 0, count = 0;

    while (i < t.length) {
        if (t[i] == p[j]) {
            i++;
            j++;
        }

        if (j == p.length) {
            System.out.println("Match at code-point index " + (i - j));
            count++;
            j = lps[j - 1];
        } else if (i < t.length && t[i] != p[j]) {
            if (j > 0) j = lps[j - 1];
            else i++;
        }
    }

    System.out.println("Total matches: " + count);
    System.out.println("Time complexity: O(n + m)");
    return count;
}

static void kmpQuery() throws IOException {
```

```
    String text = multiline("Enter your search text.");
    String pattern = read("Enter the pattern: ");
    kmp(text, pattern);
}

 MODULE 3: EDIT DISTANCE / FUZZY MATCHING

static int editDistance(String a, String b) {
    int[] x = points(a);
    int[] y = points(b);
    int[][] dp = new int[x.length + 1][y.length + 1];

    for (int i = 0; i <= x.length; i++) dp[i][0] = i;
    for (int j = 0; j <= y.length; j++) dp[0][j] = j;

    for (int i = 1; i <= x.length; i++) {
        for (int j = 1; j <= y.length; j++) {
            int cost = x[i - 1] == y[j - 1] ? 0 : 1;

            dp[i][j] = Math.min(
                    dp[i - 1][j] + 1,
                    Math.min(dp[i][j - 1] + 1,
                            dp[i - 1][j - 1] + cost));
        }
    }

    return dp[x.length][y.length];
}

static void fuzzySearch() throws IOException {
    if (docCount == 0) {
        System.out.println("Add documents first.");
        return;
```

```
}

String query = read("Enter your search word: ");
int limit = number("Maximum edit distance allowed: ");

if (limit < 0) {
   System.out.println("Distance must be non-negative.");
   return;
}

boolean found = false;

for (int d = 0; d < docCount; d++) {
   String[] words = documents[d].split("\\s+");

   for (String word : words) {
      String candidate = word.replaceAll(
            "^[\\p{Punct}]+|[\\p{Punct}]+$", "");

      if (candidate.isEmpty()) continue;

      int distance = editDistance(
            query.toLowerCase(),
            candidate.toLowerCase());

      if (distance <= limit) {
         System.out.println(
               "Document " + d + ": " + candidate +
               " | Edit distance: " + distance);
         found = true;
      }
   }
}
```

```
    if (!found) System.out.println("No similar words found.");
    System.out.println("Algorithm: Wagner-Fischer DP");
    System.out.println("Per comparison: O(n * m)");
}
```

MODULE 3: DOCUMENT SIMILARITY USING LCS

```
static void similarity() throws IOException {
    if (docCount < 2) {
        System.out.println("Add at least two documents.");
        return;
    }

    int a = number("First document ID (0 to " + (docCount - 1) + "): ");
    int b = number("Second document ID (0 to " + (docCount - 1) + "): ");

    if (a < 0 || a >= docCount || b < 0 || b >= docCount) {
        System.out.println("Invalid document ID.");
        return;
    }

    int[] x = points(documents[a]);
    int[] y = points(documents[b]);
    int[][] dp = new int[x.length + 1][y.length + 1];

    for (int i = 1; i <= x.length; i++) {
        for (int j = 1; j <= y.length; j++) {
            if (x[i - 1] == y[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
```

```
            dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    int lcs = dp[x.length][y.length];
    int sum = x.length + y.length;
    double score = sum == 0 ? 100 : 200.0 * lcs / sum;

    System.out.println("Longest common subsequence: " + lcs);
    System.out.printf("LCS similarity: %.2f%%%n", score);
    System.out.println("Time complexity: O(n * m)");
}
```

MODULE 4: EDMONDS-KARP MAXIMUM FLOW

```
static boolean bfs(int[][] r, int s, int t, int[] parent, int n) {
    boolean[] seen = new boolean[n];
    int[] q = new int[n];
    int head = 0, tail = 0;

    q[tail++] = s;
    seen[s] = true;
    parent[s] = -1;

    while (head < tail) {
        int u = q[head++];

        for (int v = 0; v < n; v++) {
            if (!seen[v] && r[u][v] > 0) {
                seen[v] = true;
                parent[v] = u;
```

```
            if (v == t) return true;
            q[tail++] = v;
          }
        }
      }

      return false;
  }

  static void maxFlowDemo() {
      int n = 6;
      int[][] c = new int[n][n];

      c[0][1] = 16;
      c[0][2] = 13;
      c[1][2] = 10;
      c[2][1] = 4;
      c[1][3] = 12;
      c[3][2] = 9;
      c[2][4] = 14;
      c[4][3] = 7;
      c[3][5] = 20;
      c[4][5] = 4;

      int[][] r = new int[n][n];

      for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
          r[i][j] = c[i][j];

      int[] parent = new int[n];
      int flow = 0;
```

```
    while (bfs(r, 0, 5, parent, n)) {
      int amount = Integer.MAX_VALUE;

      for (int v = 5; v != 0; v = parent[v])
        amount = Math.min(amount, r[parent[v]][v]);

      for (int v = 5; v != 0; v = parent[v]) {
        int u = parent[v];
        r[u][v] -= amount;
        r[v][u] += amount;
      }

      flow += amount;
    }

    System.out.println("Maximum flow: " + flow);
    System.out.println("Algorithm: Edmonds-Karp");
    System.out.println("Complexity: O(V * E^2)");
}
```

MODULE 4: BIPARTITE MATCHING

```
static boolean augment(int u, boolean[][] graph,
                       boolean[] seen, int[] match) {
    for (int v = 0; v < match.length; v++) {
      if (graph[u][v] && !seen[v]) {
        seen[v] = true;

        if (match[v] == -1 ||
          augment(match[v], graph, seen, match)) {
          match[v] = u;
          return true;
```

```
            }
        }
    }

    return false;
}

static void matchingDemo() {
    boolean[][] graph = {
            {true, true, false},
            {true, false, true},
            {false, true, true}
    };

    int[] match = {-1, -1, -1};
    int count = 0;

    for (int u = 0; u < graph.length; u++) {
        boolean[] seen = new boolean[match.length];
        if (augment(u, graph, seen, match)) count++;
    }

    System.out.println("Maximum matching size: " + count);

    for (int v = 0; v < match.length; v++)
        if (match[v] != -1)
            System.out.println("Left " + match[v] + " -> Right " + v);
}
```

MODULE 5: VERTEX-COVER 2-APPROXIMATION

```
static void vertexCoverDemo() {
```

```
    int[] u = {0, 0, 1, 1, 2, 3, 3};
    int[] v = {1, 2, 3, 4, 4, 5, 6};

    boolean[] selected = new boolean[7];

    for (int i = 0; i < u.length; i++) {
        if (!selected[u[i]] && !selected[v[i]]) {
            selected[u[i]] = true;
            selected[v[i]] = true;
        }
    }

    System.out.print("Approximate vertex cover: ");

    for (int i = 0; i < selected.length; i++)
        if (selected[i]) System.out.print(i + " ");

    System.out.println();
    System.out.println("Guarantee: at most 2 times the optimal size.");
}
```

MODULE 5: DEADLINE-ORDER SCHEDULING DEMONSTRATION

```
static void schedulingDemo() {
    String[] jobs = {"Corpus", "Index", "Search", "Testing", "Report"};
    int[] duration = {3, 2, 4, 2, 1};
    int[] deadline = {5, 4, 10, 8, 12};

    for (int i = 0; i < jobs.length - 1; i++) {
        for (int j = i + 1; j < jobs.length; j++) {
            if (deadline[j] < deadline[i]) {
```

```
            int t = deadline[i];
            deadline[i] = deadline[j];
            deadline[j] = t;

            t = duration[i];
            duration[i] = duration[j];
            duration[j] = t;

            String s = jobs[i];
            jobs[i] = jobs[j];
            jobs[j] = s;
          }
        }
    }

    int elapsed = 0;

    for (int i = 0; i < jobs.length; i++) {
        elapsed += duration[i];

        System.out.println(jobs[i] + ": finish=" + elapsed
                + ", deadline=" + deadline[i]
                + (elapsed > deadline[i] ? " (late)" : " (on time)"));
    }

    System.out.println("This is a heuristic demonstration.");
}
```

MODULE 6: MILLER-RABIN PRIMALITY TEST

```
static long addMod(long a, long b, long mod) {
    return a >= mod - b ? a - (mod - b) : a + b;
```

```
}

static long multiplyMod(long a, long b, long mod) {
    long result = 0;
    a %= mod;

    while (b > 0) {
        if ((b & 1L) != 0) result = addMod(result, a, mod);
        b >>= 1;

        if (b > 0) a = addMod(a, a, mod);
    }

    return result;
}

static long powerMod(long base, long exp, long mod) {
    long result = 1;
    base %= mod;

    while (exp > 0) {
        if ((exp & 1L) != 0)
            result = multiplyMod(result, base, mod);

        exp >>= 1;

        if (exp > 0) base = multiplyMod(base, base, mod);
    }

    return result;
}

static boolean millerRabin(long n, int rounds) {
```

```
if (n < 2) return false;
if (n == 2 || n == 3) return true;
if (n % 2 == 0) return false;

long d = n - 1;
int s = 0;

while (d % 2 == 0) {
   d /= 2;
   s++;
}

for (int r = 0; r < rounds; r++) {
   long a = 2 + (long) (Math.random() * (n - 3));
   long x = powerMod(a, d, n);

   if (x == 1 || x == n - 1) continue;

   boolean passed = false;

   for (int j = 1; j < s; j++) {
      x = multiplyMod(x, x, n);

      if (x == n - 1) {
         passed = true;
         break;
      }
   }

   if (!passed) return false;
}

return true;
```

```
}
```

MODULE 6: BLELLOCH PREFIX SCAN

```
static void prefixScan() throws IOException {
   int n = number("Number of elements: ");

   if (n < 1 || n > 100000) {
      System.out.println("Enter 1 to 100000 elements.");
      return;
   }

   long[] a = new long[n];

   for (int i = 0; i < n; i++)
      a[i] = Long.parseLong(read("Element " + i + ": ").trim());

   int size = 1;
   while (size < n) size *= 2;

   long[] tree = new long[size];

   for (int i = 0; i < n; i++) tree[i] = a[i];

   // Up-sweep.
   for (int step = 1; step < size; step *= 2)
      for (int i = 0; i < size; i += 2 * step)
         tree[i + 2 * step - 1] += tree[i + step - 1];

   tree[size - 1] = 0;
```

```
    // Down-sweep.
    for (int step = size / 2; step >= 1; step /= 2) {
        for (int i = 0; i < size; i += 2 * step) {
            long temp = tree[i + step - 1];
            tree[i + step - 1] = tree[i + 2 * step - 1];
            tree[i + 2 * step - 1] += temp;
        }
    }

    System.out.print("Prefix sums: ");

    for (int i = 0; i < n; i++)
        System.out.print((tree[i] + a[i]) + " ");

    System.out.println();
    System.out.println("Work complexity: O(n)");
    System.out.println("Sequential demonstration of a parallel algorithm.");
}



 MODULE 1: PUBLIC API



static void apiOverview() {
    System.out.println("\n=== TEXTHACK PUBLIC API ===");
    System.out.println("Document management: add and list documents");
    System.out.println("Pattern search: Aho-Corasick and KMP");
    System.out.println("Fuzzy search: edit distance DP");
    System.out.println("Document similarity: LCS DP");
    System.out.println("Network optimization: max-flow and matching");
    System.out.println("Approximation: vertex cover");
    System.out.println("Scheduling: deadline-order heuristic");
    System.out.println("Randomized algorithms: Miller-Rabin");
```

```
    System.out.println("Parallel algorithms: prefix-scan demonstration");
}

static void menu() {
    System.out.println("\n========== TEXTHACK ==========");
    System.out.println("0. API overview");
    System.out.println("1. Add your own document");
    System.out.println("2. View stored documents");
    System.out.println("3. Unicode multi-pattern search");
    System.out.println("4. KMP single-pattern search");
    System.out.println("5. Fuzzy search across documents");
    System.out.println("6. Compare two documents");
    System.out.println("7. Maximum flow");
    System.out.println("8. Bipartite matching");
    System.out.println("9. Vertex-cover approximation");
    System.out.println("10. Scheduling demonstration");
    System.out.println("11. Miller-Rabin primality test");
    System.out.println("12. Prefix scan");
    System.out.println("13. Exit");
}

public static void main(String[] args) throws IOException {
    while (true) {
        menu();

        int choice;

        try {
            choice = number("Choose an option: ");
        } catch (NumberFormatException e) {
            System.out.println("Enter a valid menu number.");
            continue;
        }
```

```
try {
   switch (choice) {
      case 0:
         apiOverview();
         break;

      case 1: {
         if (docCount >= MAX_DOCS) {
            System.out.println("Document store is full.");
            break;
         }

         String text = multiline("Enter your document.");
         if (text.isEmpty()) {
            System.out.println("Empty document not added.");
         } else {
            addDocument(text);
         }
         break;
      }

      case 2:
         showDocuments();
         break;

      case 3:
         multiPatternSearch();
         break;

      case 4:
         kmpQuery();
         break;
```

```
case 5:
   fuzzySearch();
   break;

case 6:
   similarity();
   break;

case 7:
   maxFlowDemo();
   break;

case 8:
   matchingDemo();
   break;

case 9:
   vertexCoverDemo();
   break;

case 10:
   schedulingDemo();
   break;

case 11: {
   long n = Long.parseLong(
         read("Enter a positive integer: ").trim());

   if (n < 2) {
      System.out.println("Not prime.");
   } else if (n == 2 || n == 3) {
      System.out.println("Prime.");
```

```
                } else {
                    System.out.println(
                            millerRabin(n, 10)
                                    ? "Probably prime (10 rounds)."
                                    : "Composite.");
                }

                break;
            }

            case 12:
                prefixScan();
                break;

            case 13:
                System.out.println("TextHack terminated.");
                return;

            default:
                System.out.println("Invalid option.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid number.");
        } catch (IllegalArgumentException e) {
            System.out.println("Input error: " + e.getMessage());
        }
    }
  }
}
```

RESULTS

Screenshots

```
========== TEXTHACK ==========
0. API overview
1. Add your own document
2. View stored documents
3. Unicode multi-pattern search
4. KMP single-pattern search
5. Fuzzy search across documents
6. Compare two documents
7. Maximum flow
8. Bipartite matching
9. Vertex-cover approximation
10. Scheduling demonstration
11. Miller-Rabin primality test
12. Prefix scan
13. Exit
Choose an option: 3
Add documents first.
```

```
========== TEXTHACK ==========
0. API overview
1. Add your own document
2. View stored documents
3. Unicode multi-pattern search
4. KMP single-pattern search
5. Fuzzy search across documents
6. Compare two documents
7. Maximum flow
8. Bipartite matching
9. Vertex-cover approximation
10. Scheduling demonstration
11. Miller-Rabin primality test
12. Prefix scan
13. Exit
Choose an option: 4
```

```
Enter your search text.
Type END on a separate line to finish.
I am Poojitha Rani Kandula, a B.Tech Computer Science and
Engineering student at KL Deemed to be University, with a
strong interest in software development, artificial
intelligence, and advanced algorithms. I enjoy learning new
technologies, solving problems, and developing practical
projects that connect theoretical concepts with real-world
applications. I have experience with programming languages
such as C, Java, and Python, along with web development and
database technologies. Currently, I am working on TextHack,
a Unicode text analytics engine that integrates advanced
algorithms for multi-pattern searching, fuzzy matching,
document similarity, and graph optimization. I am a
dedicated learner who values teamwork, creativity, and
continuous improvement, and I aspire to build innovative
technology solutions that make a meaningful impact.
END
Enter the pattern: meaningful
Match at code-point index 833
Total matches: 1
Time complexity: O(n + m)
```

```
========== TEXTHACK ==========
0. API overview
1. Add your own document
2. View stored documents
3. Unicode multi-pattern search
4. KMP single-pattern search
5. Fuzzy search across documents
6. Compare two documents
7. Maximum flow
8. Bipartite matching
9. Vertex-cover approximation
10. Scheduling demonstration
11. Miller-Rabin primality test
12. Prefix scan
13. Exit
Choose an option:
=== Session Ended. Please Run the code again ===
```